

Lab 10

23L-6006 Mudassar Hussain

Read

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <errno.h>

int main(int argc, char *argv[])
{
    int fd;
    char *data;

    if ((fd = open("m.c", O_RDONLY)) == -1)
    {
        perror("open");
        exit(1);
    }

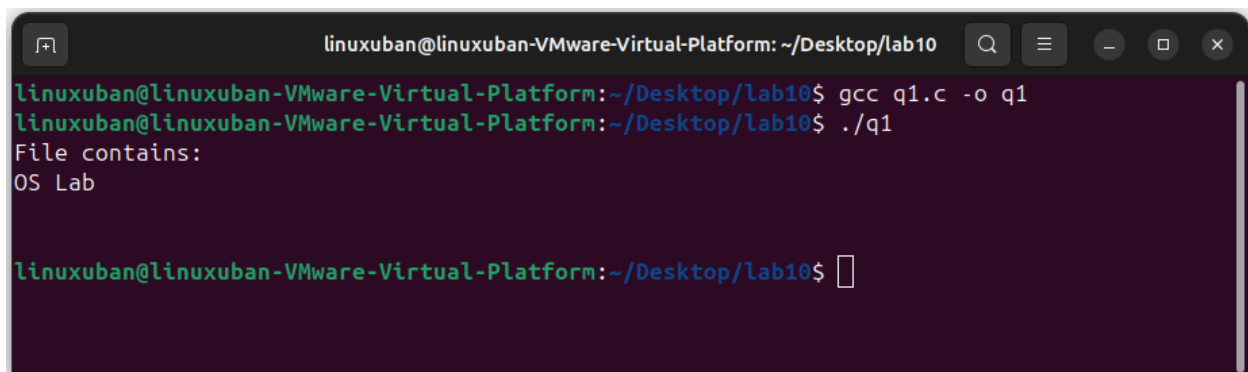
    data = mmap(NULL, getpagesize(), PROT_READ, MAP_SHARED, fd, 0);

    printf("File contains: %s\n", data);

    return 0;
}
```

M.c

OS lab

A terminal window titled 'linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab10'. The terminal shows the following commands and output:
1. 'gcc q1.c -o q1' - Compiles the file q1.c into an executable named q1.
2. './q1' - Executes the program.
3. Output: 'File contains: OS Lab'
4. The prompt returns to 'linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab10\$'.

Write

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main(int argc, char *argv[])
{
    int fd;
    void *file_memory;

    if (argc < 2) {
        printf("Usage: %s filename\n", argv[0]);
        return 1;
    }

    fd = open(argv[1], O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);

    // Set file size to 256 bytes
    ftruncate(fd, 256);

    // Memory map
    file_memory = mmap(NULL, 256, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    if (file_memory == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    close(fd);

    // Write something into mapped memory
    sprintf((char *)file_memory, "Hello from mmap!\n");

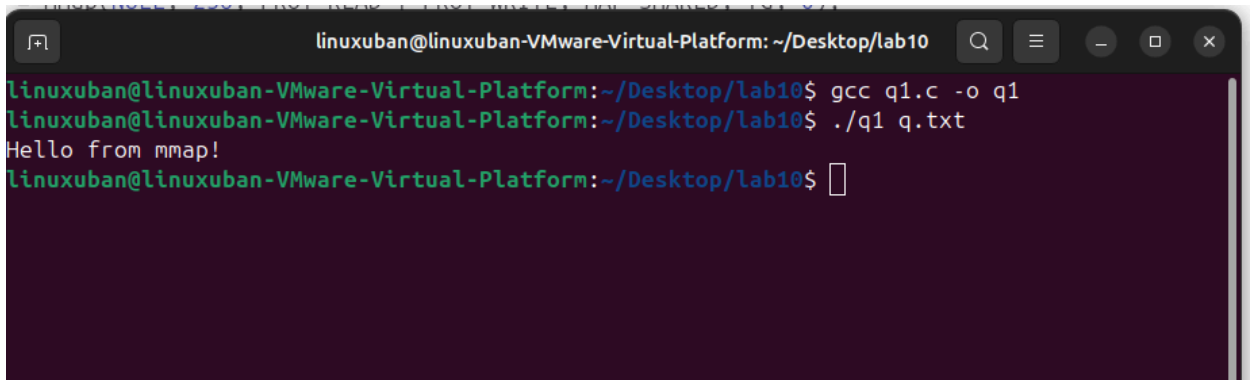
    // Read it back
    printf("%s", (char *)file_memory);
```

```

// Unmap
munmap(file_memory, 256);

return 0;
}

```



```

linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab10
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$ gcc q1.c -o q1
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$ ./q1 q.txt
Hello from mmap!
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$ 

```

Q1

```

#include <iostream>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

using namespace std;

int main(int argc, char *argv[])
{
    int fd;
    struct stat sb;
    char *data;

    // Check command line argument
    if (argc < 2)
    {
        cout << "Usage: " << argv[0] << " <filename>" << endl;
        return 1;
    }
}

```

```

// Open file
fd = open(argv[1], O_RDONLY);
if (fd == -1)
{
    perror("open");
    return 1;
}

// Get file size
if (fstat(fd, &sb) == -1)
{
    perror("fstat");
    close(fd);
    return 1;
}

// Memory map
data = (char *)mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
if (data == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return 1;
}

// Print first 50 characters (or less if file is smaller)
int limit = (sb.st_size < 50) ? sb.st_size : 50;

for (int i = 0; i < limit; i++)
{
    cout << data[i];
}
cout << endl;

// Unmap and close
munmap(data, sb.st_size);
close(fd);

return 0;
}

```

```
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab10
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$ g++ q1.cpp -o q1
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$ ./q1 test.txt
Hello! This is a simple test file for memory-mapped
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$
```

Q2

```
#include <iostream>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <cstring>

using namespace std;

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        cout << "Usage: " << argv[0] << " <filename>" << endl;
        return 1;
    }

    // Open file with read + write permission
    int fd = open(argv[1], O_RDWR);
    if (fd == -1)
    {
        perror("open");
        return 1;
    }
}
```

```

// Get file size
struct stat sb;
if (fstat(fd, &sb) == -1)
{
    perror("fstat");
    close(fd);
    return 1;
}

if (sb.st_size < 10)
{
    cout << "File too small to modify 10 characters" << endl;
    close(fd);
    return 1;
}

// Memory map with write access
char *data = (char *)mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);
if (data == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return 1;
}

// Text to write (10 characters exactly)
const char *text = "Hi There!!";

// Copy into mapped memory
memcpy(data, text, 10);

// Unmap and close
munmap(data, sb.st_size);
close(fd);

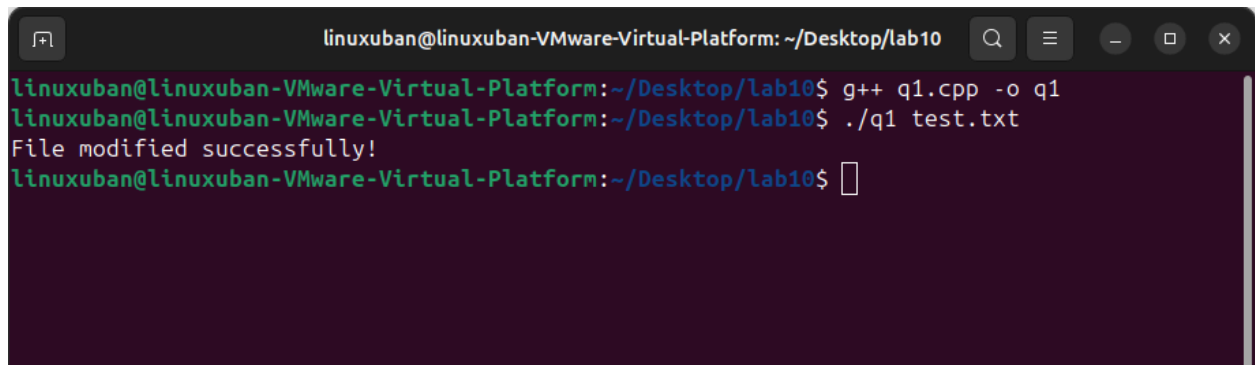
cout << "File modified successfully!" << endl; //First 10 characters

return 0;
}

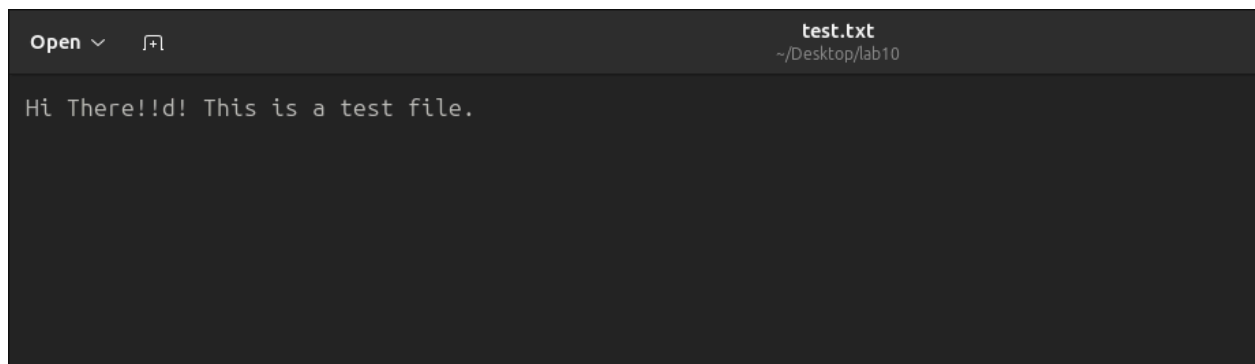
```



A screenshot of a text editor window. The title bar shows 'test.txt' and the file path '~/Desktop/lab10'. The editor contains the text 'Hello World! This is a test file.'



```
linuxuban@linuxuban-VMware-Virtual-Platform: ~/Desktop/lab10
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$ g++ q1.cpp -o q1
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$ ./q1 test.txt
File modified successfully!
linuxuban@linuxuban-VMware-Virtual-Platform:~/Desktop/lab10$
```



A screenshot of a text editor window, similar to the first one. The title bar shows 'test.txt' and the file path '~/Desktop/lab10'. The editor contains the updated text 'Hi There!!d! This is a test file.'

Q3

```
#include <iostream>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <cstring>

using namespace std;

int main(int argc, char *argv[])
{
    if (argc < 3)
    {
        cout << "Usage: " << argv[0] << " <filename> <substring>" << endl;
        return 1;
    }

    const char *filename = argv[1];
    const char *substr = argv[2];

    // Open file
    int fd = open(filename, O_RDONLY);
    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    // Get file size
    struct stat sb;
    if (fstat(fd, &sb) == -1)
    {
        perror("fstat");
        close(fd);
        return 1;
    }

    if (sb.st_size == 0)
    {
        cout << "File is empty" << endl;
        close(fd);
    }
}
```



```

    return 1;
}

// Memory map
char *data = (char *)mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
if (data == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return 1;
}

int count = 0;
int sub_len = strlen(substr);

// Search substring
for (int i = 0; i <= sb.st_size - sub_len; i++)
{
    if (strncmp(&data[i], substr, sub_len) == 0)
    {
        count++;
    }
}

// Output result
cout << "Occurrences: " << count << endl;

// Cleanup
munmap(data, sb.st_size);
close(fd);

return 0;
}

```

